Supplementary Materials

Appendix A. Computational Framework

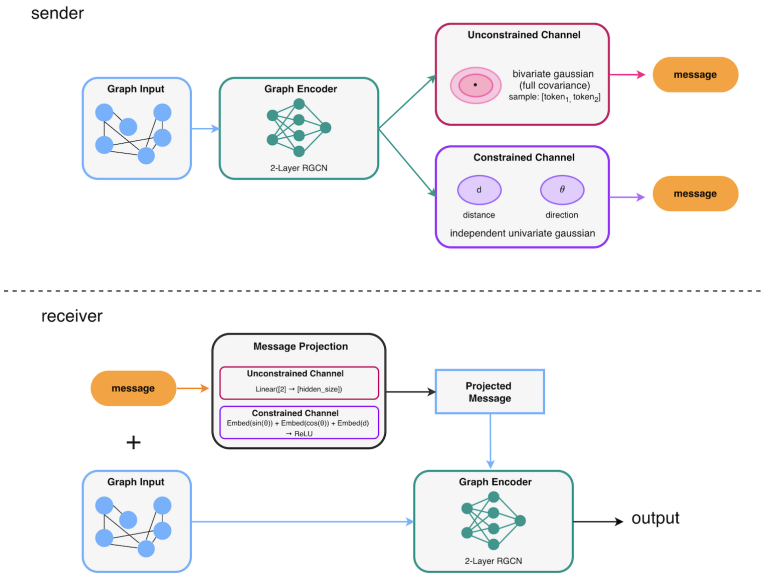
Below we introduce a computational framework to investigate whether neural agents in a foraging environment develop a communication protocol with features of the honeybee waggle dance. Our code will be publicly released in `https://github.com/nabilasiregar/BEECOM`. Our implementation is built in Python 3.11 using PyTorch to build the neural agents and the continuous communication channel. PyTorch Geometric (Fey & Lenssen, 2019) was used for the graph neural networks, and the Emergence of lanGuage in Games (EGG) library (Kharitonov, Chaabouni, Bouchacourt, & Baroni, 2019) serves as a training wrapper to orchestrate the game and manage the training loop. Each experiment was conducted five times with different random seeds (27, 31, 42, 123, 2025). Detailed agent hyperparameters are reported in Table 1.

Table 1. Hyperparameter settings

Parameter	
Batch size	32
Learning rate	0.0007
Dropout	0.1
Number of epochs	50
Sender/Receiver hidden channels	32
Sender/Receiver embedding size	32

As seen in Figure 1, we implemented a referential communication game where sender and receiver agents process graph inputs through a graph encoder before engaging in communication. The graph encoder consists of a two-layer RGCN with mean aggregation that is extended to incorporate edge weights as multiplicative scalars during message passing (details in Appendix A.3). The encoder applies ReLU activation and dropout layer after the first layer. The sender agent then maps its encoded graph representation to the continuous communication channel. We have two variants of communication channel: (i) unconstrained: where tokens are sampled jointly through a bivariate Gaussian with its full covariance matrix, (ii) constrained: sampled independently through a univariate Gaussians with diagonal covariance. Messages are sampled using the reparameterization trick to enable gradient backpropagation, while deterministic means are used during evaluation. The receiver agent processes the message through message projection layers before combining it with its own encoded graph representation. The training objective is the negative log-likelihood, with the gradients flowing through the continuous communication channel. The receiver's output are probabilities over which landmark is the target landmark.

Figure 1. Computational framework for emergent communication of the honeybee waggle dance. The sender encodes its observation into a message, the receiver decodes the message to produce an action (output probabilities over nodes).



Appendix A.1. *Game Design*

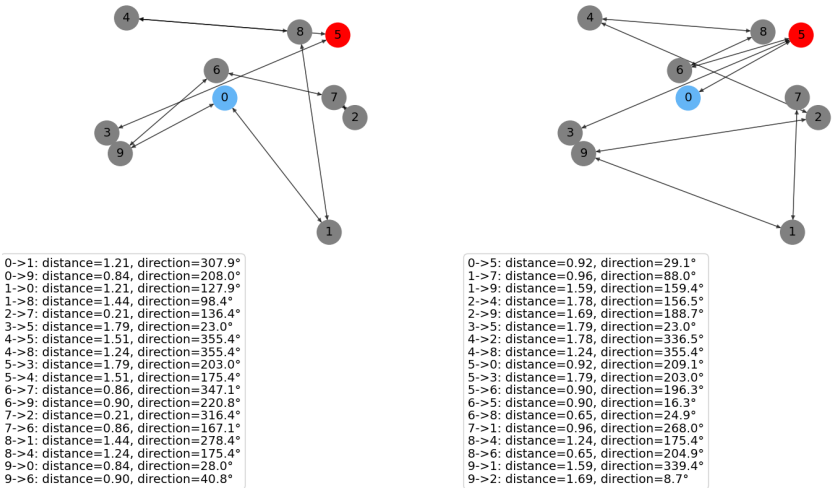
Our game design is a a form of referential game with two agents, a sender and a receiver, that are collaborating so the receiver can effectively identify the target landmark (food) from a set of landmarks in a graph-based environment. The sender's objective is to communicate a message that helps the receiver to identify the target landmark. The sender has complete knowledge of the graph, including the positions of both the food and nest within the graph. However, the receiver is only aware of the location of the nest and must rely on messages from the sender, along with locally observed edge attributes (distance and direction) to make decisions.

Appendix A.2. *Map Representation*

The foraging environment is represented as a graph where each edge encodes continuous values of distance and direction akin to the honeybee waggle dance. In honeybee waggle dance, distance is encoded by the repetition of the dance while the direction from nest to the food location is encoded by its dancing angle.

We represent each environment as a random bidirectional graph with N nodes that represent landmarks (see Figure 2). Each node is assigned one of three types:

Figure 2. Environment: a sample of independent-graph-v1 initialized with a total of 10 nodes. Sender's graph (left) and receiver's graph (right). The red node is the food landmark, the blue node denotes the nest, and grey nodes represent distractor landmarks.



nest, food, or distractor. The algorithm to generate the random graph work as follows. We first set the size of the world (a bounded rectangular space with coordinates). The nest is fixed at the center of this world and the remaining nodes are placed by sampling uniform random positions within the world. For $N = 2$, the graph consists of two nodes (nest and food) connected by bidirectional edges. For $N > 2$, we shuffle the node indices and connect them so that every node at least has one edge. From this, we add or remove edges to match one of the following cases: (i) a single route tree with a direct nest-food edge, (ii) multiple routes to reach the food node from the nest node with at least one route is a direct nest-food edge, (iii) multiple routes where a direct nest-food edge may or may not exist, (iv) a single route tree to reach food node from the nest node and it is not a direct edge. If the generated random graph doesn't meet one of the cases, the algorithm will try to add a missing edge or break a cycle and retry as needed. We then make the graph bidirectional by including both directions for every undirected edge. Edge attributes are derived from node positions: for each edge connecting nodes i and j, we compute the Euclidean distance between their coordinates and the angular direction from node i to j.

We generate four types of graph inputs that differ in edge connections but always share the same node positions: (i) same-graph: sender and receiver have identical graphs; (ii) independent-graph-v1: graphs are generated independently

with potentially overlapping edges; (iii) independent-graph-v2: graphs are generated independently, allowing only direct nest-food overlap (case (i), if any); and (iv) different-graph: sender and receiver have a different graph with no overlap edges. In our setup, the coordinates are only used to generate the graph, which then discarded during training to force the agents to reason over the graph using the edge attributes.

Appendix A.3. *Graph Encoder*

To build the agent encoder we adapt the Relational Graph Convolutional Network (RGCN) architecture (Schlichtkrull et al., 2018) so the agent can encode the graph-structured environment. The standard RGCN updates a node's representation h_i by aggregating messages from its neighbors $\mathcal{N}_i$ across a set of discrete relation types $\mathcal{R}$:

$$h_i^{(l+1)} = \sigma\left(\sum_{r\in\mathcal{R}}\sum_{j\in\mathcal{N}_i^r}\frac{1}{c_{i,r}}W_r^{(l)}h_j^{(l)} + W_0^{(l)}h_i^{(l)}\right) \tag{1}$$

This formulation requires distinct weight matrices (W_r) for each discrete relation. In our environment, edges are defined by two attributes: angular direction and distance, both are continuous values. A naive application of RGCN would require discretizing both edge attributes and creating a unique relation for each direction-distance pair, which would lead to a combinatorial explosion of relation types and loss of information.

To address this problem, we propose a modification that integrates both edge attributes directly into the message passing mechanism. We treat directions as the core relation types ($r \in \mathcal{R}$) by discretizing them to a specified number of angular bins. This discretization reflects the angular resolution of the waggle dance and allows the model to learn a unique transformation matrix W_r for each direction. The continuous edge distance d_{ij} is incorporated as a multiplicative scalar that weights the message passed from node j to i.

The RCGN layer is implemented with PyG's FastRGCNConv with the default aggregation method mean. The general update rule for a node's representation h_i at layer (l) aggregates messages from its neighbors $\mathcal{N}_i$ and combines them with a self-representation:

$$h_i^{(l+1)} = \sigma\left(\text{AGG}\left(\left\{W_{r_{ij}}^{(l)}h_j^{(l)} \mid j\in\mathcal{N}_i\right\}\right) + W_0^{(l)}h_i^{(l)}\right) \tag{2}$$

To stabilize training and having a consistent scale, the raw distances within each graph are normalized to the range $[0, 1]$:

$$\hat{d}_{ij} = \frac{d_{ij} - \min(d)}{\max(d) - \min(d) + \epsilon} \tag{3}$$

where $\min(d)$ and $\max(d)$ are the minimum and maximum distances in the graph, and ϵ is a small constant (`1e-8`) for numerical stability.

This normalized distance, $\hat{d}_{ij}$, then scales the message passed from node j to node i. Our modified update rule, using mean aggregation, becomes:

$$h_i^{(l+1)} = \sigma\left(\frac{1}{|\mathcal{N}_i|}\sum_{r\in\mathcal{R}}\sum_{j\in\mathcal{N}_i^r}\hat{d}_{ij}\cdot W_r^{(l)}h_j^{(l)} + W_0^{(l)}h_i^{(l)}\right) \tag{4}$$

This formulation allows the model to learn distinct, direction-specific transformations whose influence is continuously and dynamically scaled by the normalized distance between nodes. This approach preserves the informational integrity of both edge attributes while remaining computationally efficient.

Appendix A.4. *Communication Channel*

Unconstrained The sender agent produces a hidden representation that is linearly transformed into the mean and covariance of a multivariate normal distribution. The covariance matrix is parameterized via Cholesky decomposition, where the diagonal elements are passed through softplus activations to guarantee non-negative values. During training, both message dimensions are sampled from the same distribution using the reparameterization trick (Kingma & Welling, 2014). We do not sample at inference time and output the mean. The receiver then linearly projects the incoming messages into its input space.

Constrained Unlike the unconstrained communication channel, the direction and distance components are sampled from independent univariate normal distributions. The direction sample is passed through a sigmoid activation function and scaled to range $[0°, 360°)$, while the distance sample passes through softplus because distance is a non-negative value. The receiver decomposes messages into the sine and cosine of the angle, along with distance, embeds each component separately, and combines them with ReLU activation before processing. This channel variant also use the reparameterization trick (Kingma & Welling, 2014) during training for gradient flow and output the mean at inference time.

Appendix A.5. *Training*

We train the agents with negative log-likelihood (NLL) over the receiver's *log-softmax* outputs. For each sample i the loss is

$$NLL \;=\; -\log p_{\text{true},i} \tag{5}$$

where $p_{\text{true},i}$ is the probability (after softmax) that the receiver assigns to the correct food node. During training, we minimize the mean NLL to encourage the

predicted distribution to assign high probability on the correct node. Classification accuracy is tracked as an evaluation metric, but it is not directly part of the loss function that the model is optimizing during training.

Appendix A.6. *Input Representation*

Each node is represented by a 3-value one-hot vector that represents whether it is the nest, the food or a distractor. Each directed edge carries two attributes: the Euclidean distance between two nodes and a continuous angle value that represents the direction from one node to another node (see Figure 2). During training, we feed the node vectors and these two edge attributes to a two-layer RGCN as described in Appendix A.3. The integer indices of the nest and food nodes are stored in an auxiliary input which is used by the sender to combine the nest and food embeddings to form its message. The receiver, which never sees the food index, subtracts the nest embedding from every other node embedding, so that all information it processes is expressed relative to the nest. This setup mirrors the intuition of honeybee waggle dance where the sender agent knows both nest and food locations, whereas the receiver agent starts from the nest and infers the food location from the communicated message.

References

Fey, M., & Lenssen, J. E. (2019). Fast Graph Representation Learning with PyTorch Geometric. In *ICLR 2019 Workshop on Representation Learning on Graphs and Manifolds.* New Orleans, USA.

Kharitonov, E., Chaabouni, R., Bouchacourt, D., & Baroni, M. (2019). EGG: a toolkit for research on Emergence of lanGuage in Games. In S. Padó & R. Huang (Eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP): System Demonstrations* (pp. 55–60). Hong Kong, China: Association for Computational Linguistics.

Kingma, D. P., & Welling, M. (2014). Auto-Encoding Variational Bayes. In *Proceedings of the International Conference on Learning Representations (ICLR) 2014.* Banff, Canada.

Schlichtkrull, M., Kipf, T. N., Bloem, P., Berg, R. van den, Titov, I., & Welling, M. (2018). Modeling Relational Data with Graph Convolutional Networks. In A. Gangemi et al. (Eds.), *The Semantic Web: 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, Proceedings* (Vol. 10843, pp. 593–607). Springer, Cham.